Algorithms for genomic data analysis
Assignment 1: read mapping

winter semester 2026/2027

Task

Implement a read mapping algorithm that

- is designed to work on reads of length $\sim 1kbp$ with error rate $5 - 10\%$,
- may fail to map some reads, but should avoid incorrect alignments,
- is efficient (i.e. fast) and has good quality (i.e. high proportion of mapped reads).

Your program can be based on any approach to the approximate string matching problem.

Specification

Minimum performance requirements are following: given the reference sequence of length $\leq 20Mbp$, a collection of r reads should be processed in at most $100 + 3r$ seconds (on students server) and memory $< 1GB$, resulting in mapping $\geq 80\%$ of mappable reads and $\leq 1\%$ reads mapped incorrectly.

It should be assumed that:

- Majority of reads come from the reference sequence, but contain errors (substitutions, insertions and deletions of single nucleotides) resulting from the sequencing process.
- Errors occur independently at each position at the assumed error rate, so the total number of errors may slightly exceed 10% of the read length.
- Input data may contain *unmappable* reads that cannot be mapped to the reference sequence with error rate $< 20\%$.

A read included in the output file is considered to be correctly mapped if it is mappable and the reported mapping coordinates differ from the actual coordinates of the fragment it comes from by $\leq 20bp$.

Program should be executable using syntax:

```
python3 mapper.py reference.fasta reads.fasta output.txt
```

Input data are in fasta format. Output file should consist of one line for each mapped read, consisting of read identifier, start and end positions of read alignment, separated by tabs.

Terms and conditions

The assignment can be completed individually or in 2- or 3-person teams. Schedule:

- Submit your team by email to *dojer@mimuw.edu.pl* till November 3.
- Submit your solution to moodle till November 22.
- Present your approach in class on December 15.

In your program code you can use:

- code fragments from classes,

- libraries included in standard Python distribution, as well as NumPy, SciPy and Biopython,
- available code for building suffix arrays or similar structures (e.g. the attached implementation of the Karkkainen-Sanders algorithm).

In your program you cannot use:

- programs and libraries to read assembly, mapping, alignment, etc.
- multiprocessing commands,
- subprograms written in other languages,
- commands using jit compilers.

For a fair comparison, please avoid submitting your solution with a packaged virtual environment.

The use of artificial intelligence is subject to the following restrictions:

- the scope of AI usage should be reported in the submitted solution,
- you are responsible for the verification of the correctness of the submitted solutions and for the exposition of the approach used; in particular, you must be ready to prove you understand the methods used, the code, and the results obtained.

The solution should include:

- program file written in Python 3,
- slides describing the approach used and (if applicable) a brief summary of the use of AI,
- file with the list of AI prompts (if applicable).

Grading criteria

Points are awarded for

- meeting minimum requirements and deadlines: up to **3 points**,
- mapping correctness:

  3 points no incorrectly mapped reads,
  2 points $\leq 0.2\%$ incorrectly mapped reads,
  1 point $\leq 0.5\%$ incorrectly mapped reads,

- mapping completeness:

  3 points $\geq 99\%$ reads mapped,
  2 points $\geq 95\%$ reads mapped,
  1 point $\geq 90\%$ reads mapped,

- mapping time:

  3 points ≤ 0.2 second per read,
  2 points ≤ 0.5 second per read,
  1 point ≤ 1 second per read,

- presentation quality: up to **3 points**,
- team size:

  2 points 1 person,
  1 point 2 persons,

with the total number of points **not exceeding 15**.

Attached files

File package 1 contains:

- Python implementation of the Karkkainen-Sanders algorithm,
- example mapping program (not good enough to meet the minimum requirements),
- small example input and output files.

File package 2 contains full-size example input and output files.